

Evolved Neural Controllers vs Hand-Designed Rules for Stigmergic Sorting

Abstract

1 Introduction

Collective behaviour in social insects presents a puzzle that has resisted simple explanation for over a century: how do individuals with no global view, no central coordinator, and no shared plan produce outcomes that look designed? A colony of ants sorting its brood into colour-coded piles seems to *know* something that no single ant knows. The answer, as Grassé [5] observed, lies not in the agents but in the environment they leave behind — each moved pellet, each deposited pheromone trace, becomes a signal that biases the next action. The work is distributed across time and space, embedded in the world itself. This is stigmergy.

What attracted us to this problem is not that it is well understood, but that it sits at an unresolved tension between two very different ways of engineering collective behaviour. On one side stand hand-designed rules like Deneubourg’s probabilistic pickup/drop model [2]: elegant, minimal, provably effective for a narrow class of tasks. On the other stand evolved neural controllers: opaque, data-hungry, but capable of discovering strategies that no human would specify. The question is not which approach is “better” in the abstract — it is which one works better *here*, for this task, under these constraints, and what the answer tells us about the relationship between simplicity and learned complexity in collective systems.

We set out to test a specific hypothesis: that evolved neural controllers can discover sorting strategies that outperform the canonical Deneubourg baseline, but only under certain environmental conditions. Our intuition comes from two observations. First, the Deneubourg rule is remarkably efficient — it sorts 200 pellets into coherent clusters using nothing but local similarity counts and two parameters. Beating it is not trivial. Second, the rule is also rigid — it cannot adapt its behaviour based on what an agent is currently carrying, or modulate its exploration strategy in response to global progress. A neural controller with access to richer state information might exploit these blind spots, but only if the evolutionary budget is sufficient to discover the relevant structure.

We do not expect a single answer. If evolution beats the baseline, the finding is that learned controllers can extract value from state-dependent action selection in collective tasks — a result that generalises to other swarm robotics problems. If the baseline holds or wins, the finding is equally interesting: that minimal hand-designed rules with good parameter choice remain highly efficient, and that evolution under tight computational budgets struggles to discover them. This would be a direct empirical line back to Brooks’s argument for “intelligence without representation” [1] — that embodied, reactive controllers can outperform learned policies when the environment provides sufficient structure.

Either outcome is informative. The boundary between them — the regime in which each approach dominates — is the finding we are after.

2 Background

2.1 Stigmergic Sorting

The term *stigmergy* was coined by Grassé [5] to describe indirect coordination among social insects through environmental modifications. Workers leave traces (pheromones, moved objects) that stimulate subsequent actions by themselves or neighbours, producing emergent collective behaviour without central control.

Deneubourg et al. [2] formalised this into a computational sorting model: mobile agents pick up items probabilistically based on local similarity, carrying them until the surrounding environment matches the carried item’s type. The pickup probability decreases with the number of similar neighbours (isolated items are preferred), while the drop probability increases with local similarity (items are deposited near matching clusters). Over time, this simple rule produces globally sorted configurations from random initial placement.

The model was motivated by observations of ant brood sorting [4], where worker ants organise nymphs into colour-coded piles using only local cues. This biological grounding makes stigmergic sorting a natural testbed for comparing hand-designed behavioural rules against evolved controllers: the task is simple enough that minimal rules succeed, yet complex enough that learned policies might discover superior strategies.

2.2 Neuroevolution in Collective Tasks

3 Methods

3.1 Environment

We model a two-dimensional continuous arena of 100×100 units with hard-wall boundaries. The environment contains 200 pellets—100 red and 100 blue—placed uniformly at random. Twenty agents navigate the arena, each equipped with a circular sensor of radius $r = 5$ that reports the count of pellets per colour within range.

The action space is discrete with five options: *move* forward one unit in the current heading, *turn left* (-30°), *turn right* ($+30^\circ$), *pick up* a nearby pellet (within distance 1.0), and *drop* a carried pellet. Each agent can carry at most one pellet at a time.

We chose a 100×100 continuous arena rather than the 30×30 discrete grid used in the original Deneubourg formulation, because the larger space gives agents room for meaningful cluster formation without trivially saturating the environment. The discrete five-action space was selected over continuous heading control to keep the neural network output simple (five-way softmax) and the genetic algorithm search space well-behaved.

Simulation proceeds in synchronous discrete time: all agents compute actions based on the state at tick t , then all positions update simultaneously to $t+1$. Episodes run for at most 10,000 ticks, with early termination if cluster purity plateaus for 2,000 consecutive ticks ($\Delta < 0.01$).

All randomness is deterministic and reproducible. A master seed is split into independent subsystem streams (pellet placement, agent initialization, baseline stochasticity, genetic algorithm) via a `SeedBank` class. No global random state is used anywhere in the codebase.

3.2 Baseline: Deneubourg Model

Our baseline agent implements Deneubourg’s probabilistic pickup/drop rules [2]. Each agent carries at most one pellet and decides its action each tick based on local sensor readings.

When *not carrying* a pellet, the agent computes f as the fraction of the dominant colour among pellets within sensor radius r . The probability of attempting pickup is:

$$P_{\text{pickup}} = \left(\frac{k_1}{k_1 + f} \right)^2$$

When f is high (homogeneous neighbourhood), pickup is unlikely — the agent avoids breaking up existing clusters. When f is low (mixed neighbourhood), pickup is more likely — the agent targets isolated pellets. If pickup does not trigger, the agent performs a random walk (MOVE, TURN_LEFT, or TURN_RIGHT with equal probability).

When *carrying* a pellet, the agent computes f as the fraction of *similar-coloured* pellets in the sensor window. The probability of dropping is:

$$P_{\text{drop}} = \left(\frac{f}{k_2 + f} \right)^2$$

High f (many matching neighbours) increases drop probability — the agent deposits the pellet near a matching cluster. Low f suppresses dropping — the agent continues searching. If drop does not trigger, the agent random-walks.

We use $k_1 = 0.1$ and $k_2 = 0.3$, matching the starting values recommended in the original formulation. Sensor radius $r = 5$ units. These parameters were chosen as conventional defaults before any tuning; they represent the “off-the-shelf” Deneubourg configuration against which evolved controllers are compared.

3.3 Evaluation Metrics

We use two complementary metrics to quantify sorting performance.

Cluster Purity. Pellets are clustered using DBSCAN [3] with $\varepsilon = 5.0$ and `min_samples=2`, matching the agent sensor radius. For each non-noise cluster, purity is the fraction of pellets sharing the most common colour. The overall score is the arithmetic mean across all non-noise clusters, yielding a value in $[0, 1]$ where 1.0 means every cluster is monochromatic.

Cluster Count. The number of non-noise DBSCAN clusters. Lower values indicate more consolidation; however, cluster count alone is insufficient because an agent could achieve a low count by creating one large mixed cluster. Used alongside purity, it distinguishes “many pure clusters” from “few impure ones”.

Both metrics were validated against three synthetic configurations: (i) perfectly sorted (purity = 1.00, clusters = 2), (ii) uniformly random (purity ≈ 0.76 , clusters ≈ 41), and (iii) 80% clustered with 20% scattered noise (purity ≈ 0.78 , clusters ≈ 7). All values fell within $\pm 5\%$ of expected bounds.

3.4 Evolved Controller

4 Results

4.1 Experiment 1: Baseline vs Evolved

4.2 Extensions

5 Discussion

6 Limitations

7 Conclusion

AI Declaration

References

- [1] R. A. Brooks. Intelligence without representation. *Artificial Intelligence*, 47(1–3):131–159, 1991.
- [2] J. L. Deneubourg, S. Aron, S. Goss, and J. M. Pasteels. The dynamic of collective sorting: Robot-like ants and ant-like robots. *Proceedings of the Second Advanced Robotics Conference*, pages 83–87, 1990.
- [3] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, pages 226–231, 1996.
- [4] N. R. Franks and A. B. Sendova-Franks. Brood sorting by ants: Insights for self-organizing algorithms. *Artificial Life*, 2(4):327–347, 1992.
- [5] P.-P. Grassé. *Traité de Zoologie: Sociologie Animale*. Masson, 1959.